

ATENDELAB
Sistema de Controle de Atendimentos Acadêmicos
Projeto prático orientado por marcos — Fábrica de Software

Item	Descrição
Disciplina	Fábrica de Software
Turma	5º semestre
Tecnologias	PHP, MySQL, HTML, CSS e Bootstrap
Modelo de trabalho	Desenvolvimento individual com requisitos mínimos comuns e personalização visual/funcional controlada
Duração sugerida	5 semanas, 2 aulas por semana, 1h30 por aula

Documento-base para orientação, desenvolvimento, documentação e avaliação do projeto.

1. Apresentação do projeto

O AtendeLab é um sistema web acadêmico desenvolvido para registrar, acompanhar e consultar atendimentos realizados por professores, monitores, laboratórios, setores de apoio ou equipes acadêmicas. O projeto tem como objetivo integrar os conhecimentos de escopo, requisitos, modelagem, banco de dados, programação web, interface, documentação e entrega de software.

2. Nome do sistema

Nome-base: **AtendeLab** — *Sistema de Controle de Atendimentos Acadêmicos*.

Cada aluno poderá manter o nome AtendeLab ou criar uma variação própria, desde que preserve o objetivo principal do sistema. Exemplos: *MonitoriaLab*, *ApoioAcadêmico*, *Central de Atendimentos*, *LabHelp*, *Suporte Acadêmico*.

3. Problema que resolve

Em muitos ambientes acadêmicos, os atendimentos prestados a alunos ou à comunidade são registrados de forma informal, em cadernos, mensagens, planilhas isoladas ou sem qualquer padronização. Isso dificulta o acompanhamento histórico, a consulta por período, a identificação dos tipos de demanda mais frequentes e a geração de relatórios para tomada de decisão.

O sistema resolve esse problema ao centralizar o cadastro de pessoas atendidas, os tipos de atendimento e o registro das ocorrências em uma aplicação web simples, acessível e organizada.

4. Público-alvo

- Professores que realizam atendimento acadêmico.
- Monitores ou tutores responsáveis por tirar dúvidas.
- Laboratórios de informática e setores de suporte.
- Coordenações de curso ou núcleos de apoio.
- Alunos ou membros da comunidade que recebem atendimento.

5. Objetivo geral

Desenvolver uma aplicação web funcional para controle de atendimentos acadêmicos, utilizando PHP, MySQL, HTML, CSS e Bootstrap, contemplando página pública, login, área administrativa, cadastros, registros, filtros, dashboard, relatórios e documentação do sistema.

6. Objetivos específicos

- Aplicar conceitos de escopo, requisitos e regras de negócio em um projeto realista.
- Modelar entidades, relacionamentos e fluxos do sistema.
- Implementar autenticação simples com sessão em PHP.
- Criar CRUDs integrados ao banco de dados MySQL.
- Registrar atendimentos relacionando pessoa atendida, tipo de atendimento e usuário responsável.
- Desenvolver listagens com filtros e consultas usando SQL.
- Criar dashboard com indicadores operacionais.
- Produzir documentação técnica do backend usando OpenAPI/Swagger.
- Produzir Wiki de uso do sistema voltada ao cliente/usuário.
- Preparar o sistema para apresentação, testes e deploy simples.

7. Escopo do projeto

O escopo mínimo obrigatório define o que todo aluno deverá entregar para que o projeto seja considerado funcional.

Módulo	Descrição
Página pública	Landing page antes do login apresentando o sistema e botão de acesso.
Login	Autenticação com e-mail e senha usando sessão PHP.
Dashboard	Tela inicial da área restrita com indicadores.
Pessoas atendidas	Cadastro, listagem, edição e inativação/exclusão.
Tipos de atendimento	Cadastro de categorias de atendimento.
Atendimentos	Registro do atendimento vinculando pessoa, tipo e usuário responsável.
Filtros	Busca por pelo menos status, período ou nome da pessoa.
Relatórios	Relatório em HTML com opção de impressão.
Documentação	Swagger/OpenAPI para backend e Wiki para uso do cliente.
Banco de dados	Script SQL exportado com estrutura e dados básicos de teste.

8. Fora do escopo

- Aplicativo mobile.
- Integração com e-mail, SMS ou WhatsApp.
- Geração obrigatória de PDF.
- Controle financeiro.
- Sistema complexo de permissões.
- Recuperação real de senha por e-mail.
- Deploy em ambiente profissional com domínio próprio obrigatório.

- Uso de frameworks backend como Laravel ou frontend como Vue/React.

9. Regras de negócio

Código	Regra
RN01	Somente usuários autenticados podem acessar a área administrativa.
RN02	Todo atendimento deve estar vinculado a uma pessoa atendida.
RN03	Todo atendimento deve possuir um tipo de atendimento.
RN04	Todo atendimento deve registrar o usuário responsável pela criação.
RN05	Um atendimento pode ter os status: aberto, em andamento ou concluído.
RN06	Atendimentos concluídos devem permitir observação final.
RN07	Uma pessoa atendida pode possuir vários atendimentos.
RN08	Um tipo de atendimento pode estar associado a vários atendimentos.
RN09	Usuários inativos não devem conseguir acessar o sistema.
RN10	O relatório deve permitir consulta por período.
RN11	Registros importantes não devem ser apagados fisicamente quando houver relacionamento; preferencialmente devem ser inativados.
RN12	Campos obrigatórios devem ser validados antes de salvar no banco de dados.

10. Requisitos funcionais

Código	Requisito funcional
RF01	Exibir página pública com apresentação do sistema.
RF02	Permitir login de usuário cadastrado.
RF03	Permitir logout do usuário autenticado.
RF04	Proteger páginas internas contra acesso sem sessão.
RF05	Cadastrar pessoas atendidas.
RF06	Listar, editar e inativar/excluir pessoas atendidas.
RF07	Cadastrar tipos de atendimento.
RF08	Listar, editar e inativar/excluir tipos de atendimento.
RF09	Registrar novo atendimento.
RF10	Listar atendimentos com dados relacionados usando JOIN.
RF11	Alterar status de atendimento.
RF12	Filtrar atendimentos por status, pessoa, tipo, responsável ou período.
RF13	Exibir dashboard com indicadores.
RF14	Gerar relatório em HTML com opção de impressão.
RF15	Documentar rotas/endpoints do backend em Swagger/OpenAPI.
RF16	Criar Wiki com manual de uso para cliente/usuário.

11. Requisitos não funcionais

Código	Requisito não funcional
RNF01	O sistema deve ser desenvolvido em PHP, MySQL, HTML, CSS e Bootstrap.
RNF02	O sistema deve executar em ambiente XAMPP.
RNF03	O layout deve ser responsivo em nível básico.
RNF04	O código deve estar organizado em pastas.
RNF05	As senhas devem ser armazenadas com hash usando password_hash.
RNF06	As consultas ao banco devem utilizar PDO ou mysqli com tratamento adequado.
RNF07	O sistema deve apresentar mensagens de sucesso e erro.
RNF08	O banco deve possuir chaves primárias, estrangeiras e tipos de dados coerentes.
RNF09	O projeto deve conter documentação mínima para instalação e uso.
RNF10	O aluno deve ser capaz de explicar o funcionamento do código apresentado.

12. Diagramas obrigatórios

Os diagramas devem ser entregues como parte da documentação do projeto. Eles não precisam ser extremamente complexos, mas devem representar corretamente o sistema implementado.

12.1 Diagrama de casos de uso

Recomendado como diagrama adicional obrigatório, pois ajuda a conectar os atores às funcionalidades do sistema. Ele é simples, visual e adequado para comunicação com cliente e professor.

Diagrama de Caso de Uso AtendLav - Univille - 2026.1

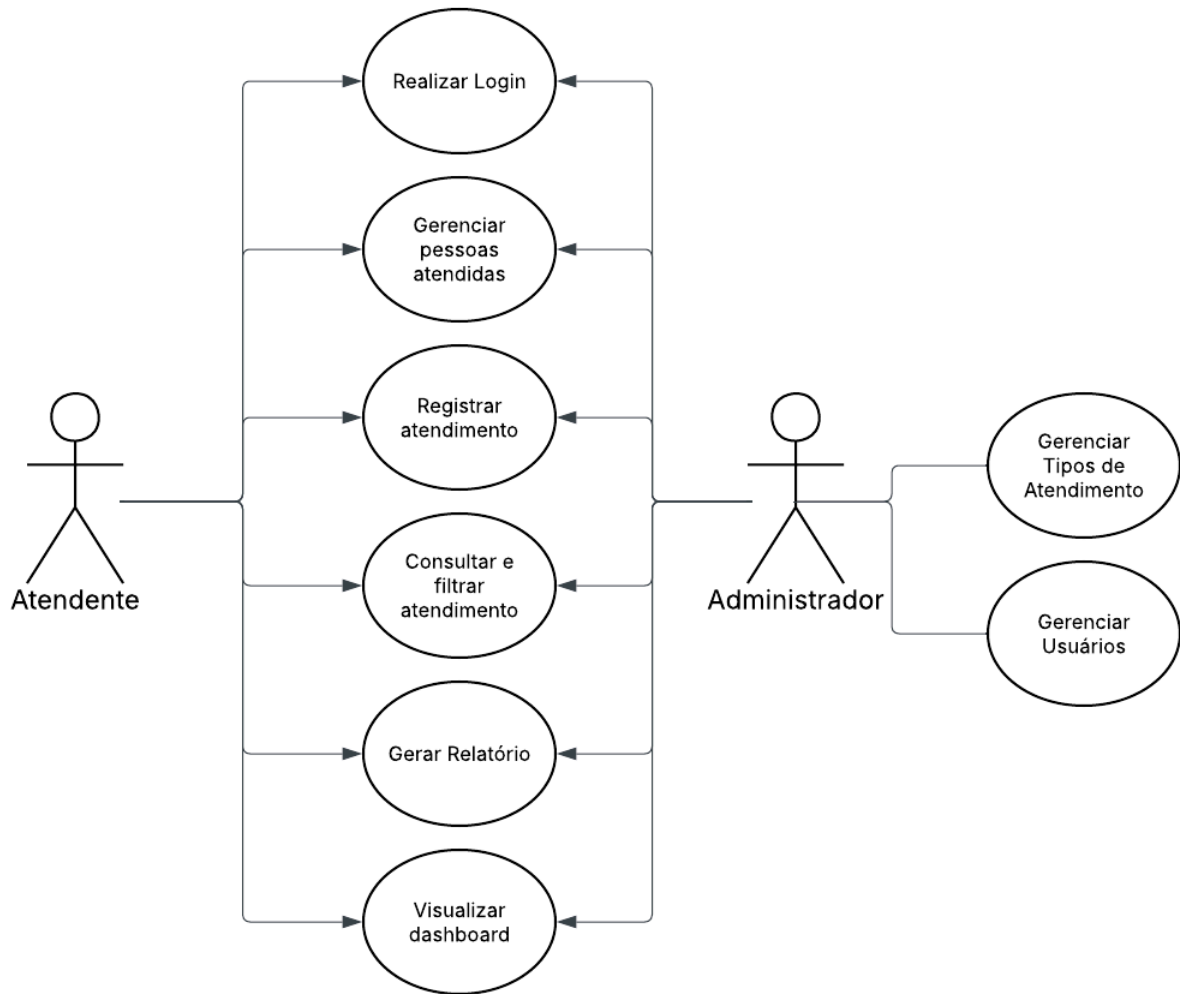


Figura 1 — Diagrama de casos de uso sugerido para o AtendeLab.

Fonte: O autor

https://lucid.app/lucidchart/c2f904af-3711-4ad1-873d-c8f5bcc7ba50/edit?viewport_loc=-92%2C-139%2C2744%2C1612%2C0_0&invitationId=inv_e53693a4-73f1-40d0-bda0-d5f5d7442cd7

12.2 Diagrama de classes

Representa as principais classes/conceitos do sistema, seus atributos e seus relacionamentos. Mesmo que o projeto use PHP procedural, o diagrama ajuda a organizar o domínio.

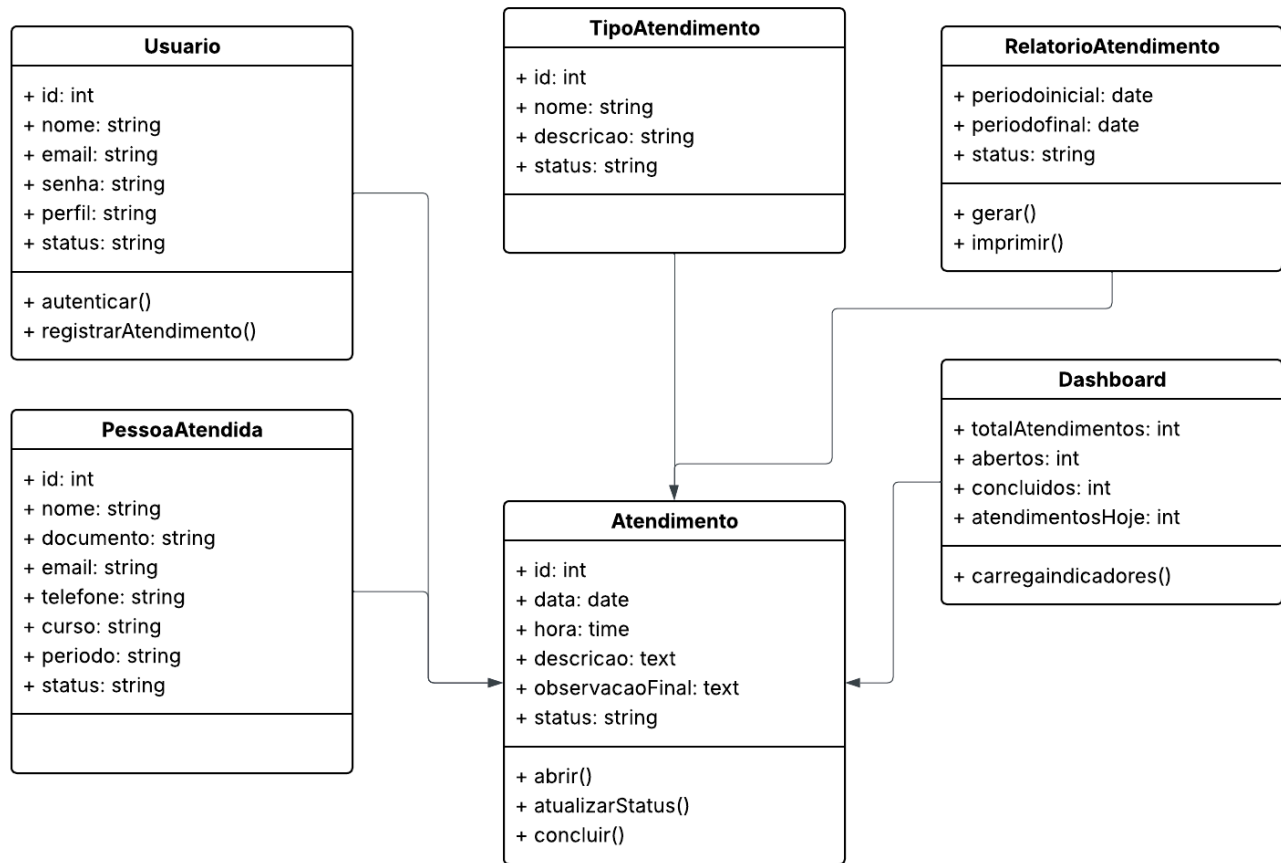


Figura 2 — Diagrama de classes conceitual do AtendeLab.

Fonte: o autor

https://lucid.app/lucidchart/84b6a309-0a8d-4506-9386-357a620ba837/edit?viewport_loc=-324%2C-186%2C2673%2C1570%2C0_0&invitation_id=inv_537f8192-ab58-42fa-bf3a-29f7dc417778

12.3 Diagrama de Entidade e Relacionamento

Representa a estrutura do banco de dados e seus relacionamentos. Este é um dos diagramas mais importantes para este projeto.

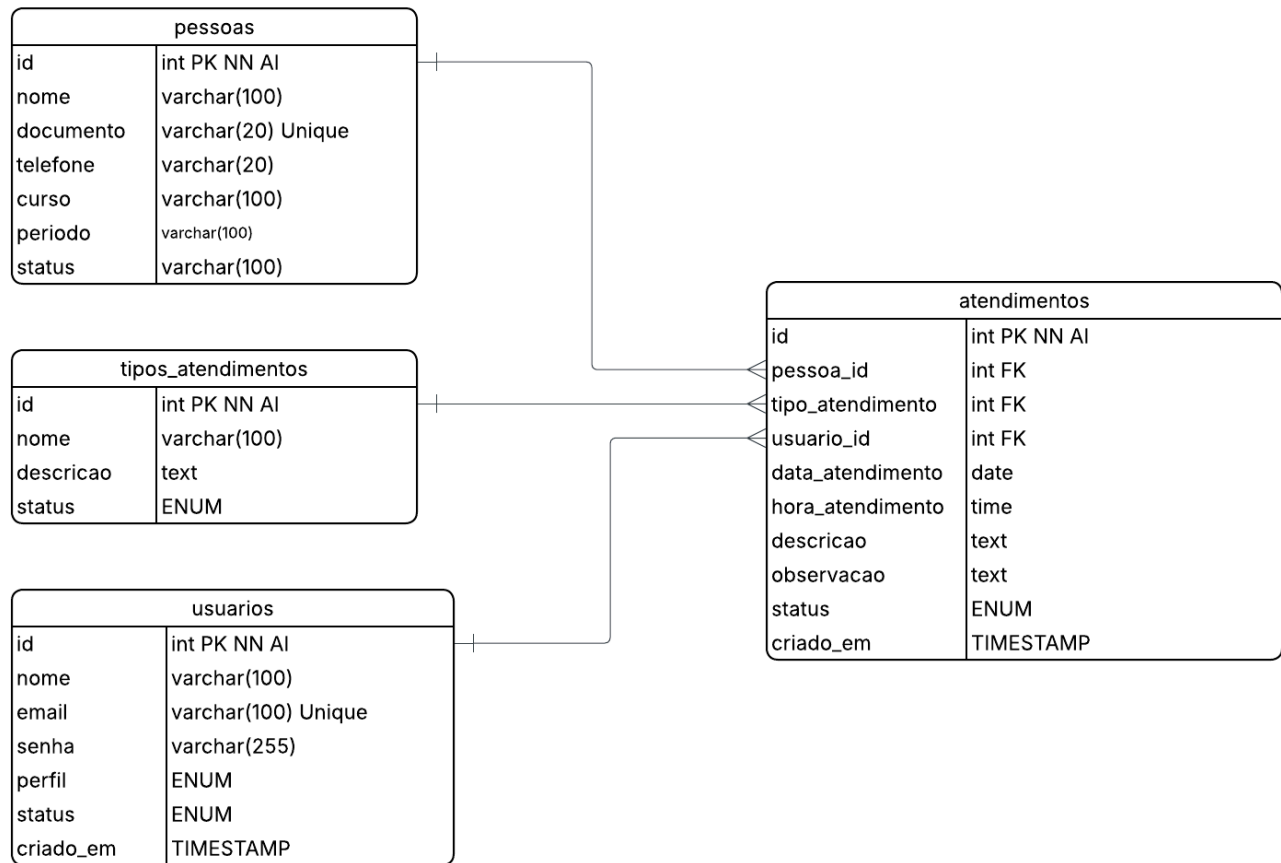


Figura 3 — Diagrama de Entidade e Relacionamento sugerido.

Fonte: o autor

https://lucid.app/lucidchart/a09bbf9b-9fbd-4525-a11a-34264ac12be3/edit?viewport_loc=-563%2C-340%2C3020%2C1774%2C0_0&invitationId=inv_83595de9-eb60-4146-837c-fc221be9e5a4

12.4 Diagrama de sequência

Representa o fluxo principal de uma operação. Para o AtendeLab, recomenda-se modelar o fluxo de registro de atendimento.

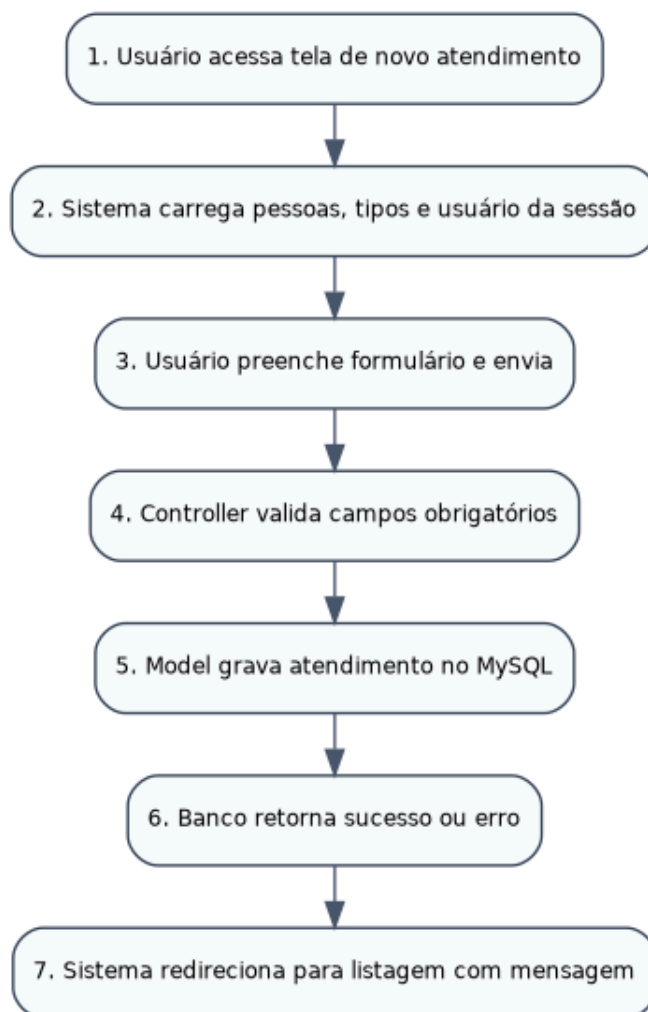


Figura 4 — Diagrama de sequência simplificado para cadastro de atendimento.

12.5 Mapa de telas

Mostra a navegação entre as páginas do sistema. É essencial para orientar a implementação e evitar telas soltas.

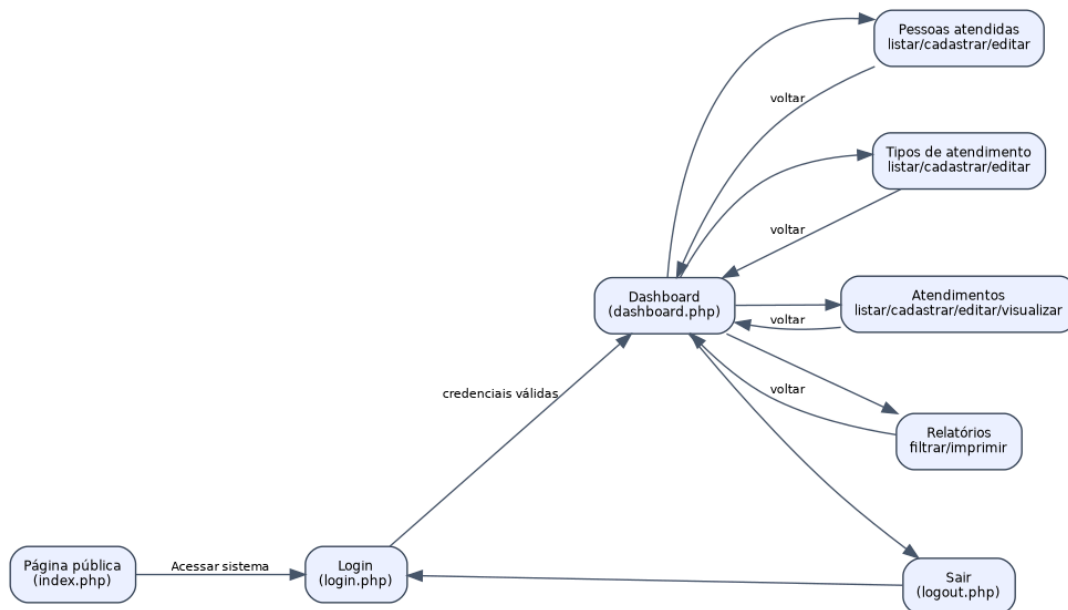


Figura 5 — Mapa de telas sugerido.

13. Outros diagramas recomendados

Além dos diagramas citados, recomenda-se incluir mais dois diagramas quando houver tempo ou quando o aluno quiser melhorar a documentação:

Diagrama	Indicação
Diagrama de casos de uso	Altamente recomendado; deve ser tratado como obrigatório no projeto, pois comunica bem o escopo funcional.
Diagrama de atividades	Opcional; útil para representar o fluxo de abertura, atendimento e conclusão de um atendimento.
Diagrama de componentes	Opcional avançado; mostra separação entre páginas, controllers, banco e documentação.
Diagrama de implantação	Opcional; útil na etapa de deploy para mostrar navegador, servidor Apache/PHP e MySQL.

14. Modelo de banco de dados

A estrutura abaixo é a base mínima sugerida. O aluno pode acrescentar campos e tabelas, desde que mantenha os relacionamentos principais.

15. Estrutura de pastas recomendada

Para o prazo disponível, recomenda-se uma estrutura simples, compreensível e próxima de um MVC básico.

```
atendelab/
├── config/
│   └── database.php
├── includes/
│   ├── header.php
│   ├── sidebar.php
│   ├── footer.php
│   └── auth.php
├── assets/
│   ├── css/
│   ├── js/
│   └── img/
├── index.php
├── login.php
├── logout.php
├── dashboard.php
├── pessoas/
│   ├── index.php
│   ├── criar.php
│   ├── editar.php
│   └── excluir.php
├── tipos/
│   ├── index.php
│   ├── criar.php
│   ├── editar.php
│   └── excluir.php
├── atendimentos/
│   ├── index.php
│   ├── criar.php
│   ├── editar.php
│   ├── visualizar.php
│   └── excluir.php
├── relatorios/
│   └── atendimentos.php
├── docs/
│   ├── openapi.yaml
│   └── wiki/
├── database/
│   └── atendelab.sql
```

16. Mapa de telas detalhado

Tela	Acesso	Finalidade
index.php	Pública	Apresentar o sistema e direcionar para o login.
login.php	Pública	Autenticar usuário.
dashboard.php	Restrita	Exibir indicadores e atalhos.
personas/index.php	Restrita	Listar pessoas atendidas.
personas/criar.php	Restrita	Cadastrar pessoa atendida.
personas/editar.php	Restrita	Editar pessoa atendida.
tipos/index.php	Restrita	Listar tipos de atendimento.
tipos/criar.php	Restrita	Cadastrar tipo de atendimento.
atendimentos/index.php	Restrita	Listar e filtrar atendimentos.
atendimentos/criar.php	Restrita	Registrar atendimento.
atendimentos/visualizar.php	Restrita	Visualizar detalhes do atendimento.
relatorios/atendimentos.php	Restrita	Gerar relatório por período.

17. Documentação técnica com Swagger/OpenAPI

A documentação do backend deverá ser criada usando o padrão OpenAPI, conhecido anteriormente como Swagger. O OpenAPI descreve endpoints, métodos HTTP, parâmetros, retornos, autenticação e modelos de dados de APIs REST. O Swagger UI permite visualizar essa documentação de forma interativa.

Como o projeto será desenvolvido em PHP simples, a documentação pode ser criada manualmente no arquivo docs/openapi.yaml. Não é obrigatório que todas as telas tenham endpoints REST reais, mas o aluno deverá documentar as principais operações do backend como se fossem recursos de API.

Método	Rota sugerida	Descrição
GET	/api/pessoas	Listar pessoas atendidas
POST	/api/pessoas	Cadastrar pessoa atendida
PUT	/api/pessoas/{id}	Atualizar pessoa atendida
DELETE	/api/pessoas/{id}	Inativar/remover pessoa atendida
GET	/api/tipos-atendimento	Listar tipos de atendimento
POST	/api/tipos-atendimento	Cadastrar tipo de atendimento
GET	/api/atendimentos	Listar atendimentos com filtros
POST	/api/atendimentos	Registrar atendimento

PUT	/api/atendimentos/{id}/status	Atualizar status do atendimento
GET	/api/relatorios/atendimentos	Gerar relatório de atendimentos

openapi: 3.0.3

info:

title: AtendeLab API

version: 1.0.0

description: Documentação das operações principais do sistema AtendeLab.

servers:

- url: http://localhost/atendelab

paths:

/api/atendimentos:

get:

summary: Lista atendimentos

parameters:

- in: query

name: status

schema:

type: string

enum: [aberto, em_andamento, concluido]

- in: query

name: data_inicio

schema:

type: string

format: date

responses:

'200':

description: Lista de atendimentos retornada com sucesso

post:

summary: Registra um novo atendimento

requestBody:

required: true

content:

application/json:

schema:

type: object

required: [pessoa_id, tipo_atendimento_id, descricao]

properties:

pessoa_id:

type: integer

```

    tipo_atendimento_id:
      type: integer
    descricao:
      type: string
  responses:
    '201':
      description: Atendimento cadastrado com sucesso

```

Referências conceituais: Swagger/OpenAPI permite descrever APIs REST, incluindo endpoints, operações, parâmetros, entradas, saídas e autenticação; Swagger UI permite visualizar definições OpenAPI em uma interface interativa.

18. Wiki do cliente/usuário

Além da documentação técnica, cada aluno deverá criar uma Wiki voltada para o cliente ou usuário final. A Wiki deve explicar como utilizar o sistema, não como o código foi programado. A Wiki pode ser criada no GitHub Wiki, em arquivos Markdown dentro da pasta docs/wiki ou em outro formato aprovado pelo professor.

A Wiki é indicada para documentação mais longa e detalhada do projeto, incluindo como usar, como configurar, princípios do sistema e orientações ao usuário.

Página da Wiki	Conteúdo esperado
Página inicial	O que é o AtendeLab e para que serve.
Como acessar	Explicar login, usuário de teste e área restrita.
Dashboard	Explicar os indicadores exibidos.
Pessoas atendidas	Como cadastrar, editar e consultar.
Tipos de atendimento	Como organizar as categorias.
Atendimentos	Como registrar, consultar, filtrar e concluir.
Relatórios	Como gerar e imprimir relatórios.
Perguntas frequentes	Dúvidas comuns e respostas.
Manual de instalação	Como instalar no XAMPP, importar banco e testar.

19. Marcos de desenvolvimento

Período	Entregas esperadas
Semana 1	Escopo, documentação inicial, banco, estrutura de pastas e página pública.
Semana 2	Login, sessão, logout, proteção de páginas e CRUD de pessoas.
Semana 3	CRUD de tipos de atendimento e registro de atendimentos com relacionamento.
Semana 4	Filtros, dashboard, relatório, Swagger/OpenAPI e Wiki.

Semana 5	Testes, correções, deploy/simulação de deploy e apresentação final.
----------	---

20. Entregas obrigatórias

- Código-fonte completo do sistema.
- Arquivo SQL exportado do banco de dados.
- Página pública antes do login.
- Área restrita com login e sessão.
- CRUDs e módulo principal de atendimento funcionando.
- Filtros, dashboard e relatório.
- Diagramas obrigatórios atualizados conforme o sistema implementado.
- Arquivo docs/openapi.yaml ou documentação Swagger equivalente.
- Wiki do cliente/usuário.
- Apresentação final demonstrando o sistema em execução.

21. Critérios de avaliação sugeridos

Critério	Peso
Escopo, requisitos e regras de negócio	1,0
Modelagem: casos de uso, classes, DER, sequência e mapa de telas	1,5
Banco de dados e relacionamentos	1,0
Login, sessão e proteção de páginas	1,0
CRUDs obrigatórios	1,5
Registro de atendimentos com relacionamento	1,5
Filtros, dashboard e relatório	1,0
Documentação Swagger/OpenAPI e Wiki	1,0
Interface, Bootstrap, usabilidade e personalização	0,8
Apresentação final e domínio do código	0,7

22. Regras sobre personalização, templates e autoria

- Todos os alunos desenvolvem o mesmo sistema-base, mas poderão personalizar nomes, cores, textos, layout e funcionalidades extras.
- É permitido utilizar Bootstrap e templates gratuitos como base visual, desde que o aluno adapte e compreenda o código utilizado.
- Não é permitido entregar sistema pronto baixado da internet.
- Não é permitido usar IA generativa para produzir código, documentação ou telas do projeto.
- O aluno deverá conseguir explicar a estrutura do banco, os principais arquivos e o fluxo de funcionamento.
- Funcionalidades extras só serão avaliadas após o MVP obrigatório estar funcionando.

23. Funcionalidades extras para alunos avançados

- Perfil de usuário admin e atendente com permissões diferentes.
- Página pública para solicitação de atendimento.
- Upload de foto ou documento.
- Exportação CSV.
- Paginação nas listagens.
- Logs de ações.
- Máscaras de CPF, telefone e datas.
- Relatório com layout específico para impressão.
- Diagrama de atividades, componentes ou implantação.
- Deploy em hospedagem PHP/MySQL ou ambiente institucional.

24. Referências de apoio

- Swagger Docs. Documentação oficial do Swagger e ferramentas relacionadas ao OpenAPI. Disponível em: <https://swagger.io/docs/>
- Swagger. What Is OpenAPI? Explicação sobre especificação OpenAPI para APIs REST. Disponível em: https://swagger.io/docs/specification/v3_0/about/
- Swagger UI. Ferramenta para visualizar documentação OpenAPI em interface interativa. Disponível em: <https://swagger.io/tools/swagger-ui/>
- GitHub Docs. About Wikis. Documentação sobre uso de Wikis para documentação de projetos. Disponível em: <https://docs.github.com/communities/documenting-your-project-with-wikis/about-wikis>
- Documentação oficial do PHP. Disponível em: <https://www.php.net/docs.php>
- Documentação oficial do MySQL. Disponível em: <https://dev.mysql.com/doc/>
- Bootstrap. Documentação oficial. Disponível em: <https://getbootstrap.com/docs/>

25. Checklist final do aluno

- ☐ Meu sistema abre corretamente no XAMPP.
- ☐ A página pública aparece antes do login.
- ☐ O login funciona com usuário de teste.
- ☐ As páginas internas estão protegidas por sessão.
- ☐ Consigo cadastrar, listar e editar pessoas atendidas.
- ☐ Consigo cadastrar, listar e editar tipos de atendimento.
- ☐ Consigo registrar atendimento usando pessoa, tipo e responsável.
- ☐ A listagem de atendimentos mostra dados relacionados corretamente.
- ☐ Existe pelo menos um filtro funcionando.
- ☐ O dashboard apresenta indicadores reais do banco.
- ☐ O relatório em HTML funciona e pode ser impresso.

- ☐ Meu banco foi exportado em arquivo SQL.
- ☐ Meus diagramas representam o sistema implementado.
- ☐ Minha documentação Swagger/OpenAPI está criada.
- ☐ Minha Wiki explica como o cliente utiliza o sistema.
- ☐ Consigo apresentar e explicar meu projeto sem depender de leitura de código pronto.